

LAYER: 2_MesCore

This file is a merged representation of a subset of the codebase, containing files not matching ignore patterns, combined into a single document by Repomix.

<file_summary>

This section contains a summary of this file.

<purpose>

This file contains a packed representation of a subset of the repository's contents that is considered the most important context.

It is designed to be easily consumable by AI systems for analysis, code review, or other automated processes.

</purpose>

<file_format>

The content is organized as follows:

1. This summary section
2. Repository information
3. Directory structure
4. Repository files (if enabled)
5. Multiple file entries, each consisting of:
 - File path as an attribute
 - Full contents of the file

</file_format>

<usage_guidelines>

- This file should be treated as read-only. Any changes should be made to the original repository files, not this packed version.
- When processing this file, use the file path to distinguish between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with the same level of security as you would the original repository.

</usage_guidelines>

<notes>

- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching these patterns are excluded: `**/obj/**`, `**/bin/**`, `**/*.png`, `**/*.jpg`, `**/node_modules/**`, `**/*.Designer.cs`
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Files are sorted by Git change count (files with more changes are at the bottom)

</notes>

</file_summary>

<directory_structure>

```
Promatis.Net.MES.Configuration/Class1.cs
Promatis.Net.MES.Configuration/Promatis.Net.MES.Configuration.csproj
Promatis.Net.MES.Data/Configuration/TechnologicalOperationParameterBaseConfiguration.cs
Promatis.Net.MES.Data/Configuration/TechnologicalOperationUnitBaseConfiguration.cs
Promatis.Net.MES.Data/Configuration/UnitBaseConfiguration.cs
Promatis.Net.MES.Data/DbContext/MesApplicationDbContext.cs
Promatis.Net.MES.Data/Promatis.Net.MES.Data.csproj
Promatis.Net.MES.Data/Triggers/TechnologicalOperationHierarchyTrigger.cs
Promatis.Net.MES.Data/Triggers/UnitBaseHierarchyTrigger.cs
Promatis.Net.MES.Domain.Interface/Enums/CalculationMethod.cs
Promatis.Net.MES.Domain.Interface/Enums/UnitKind.cs
Promatis.Net.MES.Domain.Interface/Enums/UnitType.cs
Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalOperation.cs
Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalParameter.cs
Promatis.Net.MES.Domain.Interface/Promatis.Net.MES.Domain.Interface.csproj
Promatis.Net.MES.Domain/Promatis.Net.MES.Domain.csproj
Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBase.cs
Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationHierarchyEngine.cs
Promatis.Net.MES.Domain/TechnologicalOperationParameter/TechnologicalOperationParameterBase.cs
Promatis.Net.MES.Domain/TechnologicalOperationParameter/
TechnologicalOperationParameterBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalOperationUnit/TechnologicalOperationUnitBase.cs
Promatis.Net.MES.Domain/TechnologicalOperationUnit/TechnologicalOperationUnitBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBase.cs
Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/TechnologicalParameterCalcMethodBase.cs
```

```

Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/
TechnologicalParameterCalcMethodBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalParameterValue/TechnologicalParameterValueBase.cs
Promatis.Net.MES.Domain/TechnologicalParameterValue/TechnologicalParameterValueBaseValidator.cs
Promatis.Net.MES.Domain/UnitBase/UnitBase.cs
Promatis.Net.MES.Domain/UnitBase/UnitBaseValidator.cs
Promatis.Net.MES.Domain/UnitBase/UnitHierarchyEngine.cs
Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurement.cs
Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurementValidator.cs
Promatis.Net.MES.Service/Promatis.Net.MES.Service.csproj
Promatis.Net.MES.Service/TechnologicalOperationParameterService/
ITechnologicalOperationParameterService.cs
Promatis.Net.MES.Service/TechnologicalOperationParameterService/
TechnologicalOperationParameterService.cs
Promatis.Net.MES.Service/TechnologicalOperationService/ITechnologicalOperationService.cs
Promatis.Net.MES.Service/TechnologicalOperationService/TechnologicalOperationService.cs
Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
ITechnologicalParameterCalcMethodService.cs
Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
TechnologicalParameterCalcMethodService.cs
Promatis.Net.MES.Service/TechnologicalParameterService/ITechnologicalParameterService.cs
Promatis.Net.MES.Service/TechnologicalParameterService/TechnologicalParameterService.cs
Promatis.Net.MES.Service/UnitOfMeasurementService/IUnitOfMeasurementService.cs
Promatis.Net.MES.Service/UnitOfMeasurementService/UnitOfMeasurementService.cs
Promatis.Net.MES.Service/UnitService/IUnitBaseService.cs
Promatis.Net.MES.Service/UnitService/UnitBaseService.cs
Promatis.Net.MES.Tests/AssemblyInfo.cs
Promatis.Net.MES.Tests/Promatis.Net.MES.Tests.csproj
Promatis.Net.MES.Tests/Service/UnitServiceStandaloneTests.cs
Promatis.Net.MES.Tests/UnitBase/AppDbContext.cs
Promatis.Net.MES.Tests/UnitBase/TestUnit.cs
Promatis.Net.MES.Tests/UnitBase/TestUnitValidator.cs
Promatis.Net.MES.Tests/UnitBase/UnitBaseArchitectureTests.cs
Promatis.Net.MES.UI/_Imports.razor
Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementPage.razor
Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementPage.razor.cs
Promatis.Net.MES.UI/Promatis.Net.MES.UI.csproj
Promatis.Net.MES.UI/UiModule.cs
Promatis.Net.MES.UI/wwwroot/exampleJsInterop.js
</directory_structure>

```

<files>

This section contains the contents of the repository's files.

<file path="Promatis.Net.MES.Configuration/Class1.cs">

namespace Promatis.Net.MES.Configuration

{

public class Class1

{

}

}

</file>

<file path="Promatis.Net.MES.Configuration/Promatis.Net.MES.Configuration.csproj">

<Project Sdk="Microsoft.NET.Sdk">

<PropertyGroup>

<TargetFramework>net10.0</TargetFramework>

<ImplicitUsings>enable</ImplicitUsings>

<Nullable>enable</Nullable>

</PropertyGroup>

<ItemGroup>

<ProjectReference Include="..\..\Core\Configuration.Web\Promatis.Net.Configuration.Web.csproj" /

>

<ProjectReference Include="..\Promatis.Net.MES.Service\Promatis.Net.MES.Service.csproj" />

</ItemGroup>

</Project>

</file>

<file path="Promatis.Net.MES.Data/Configuration/TechnologicalOperationParameterBaseConfiguration.cs">

using Microsoft.EntityFrameworkCore;

using Microsoft.EntityFrameworkCore.Metadata.Builders;

```

using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Data;

public abstract class TechnologicalOperationParameterBaseConfiguration<T, TOperation, TParameter> :
    IEntityTypeConfiguration<T>
    where T : TechnologicalOperationParameterBase<TOperation, TParameter> // <--- A05D 5CD0CT< C0;CäAC=8CR ?C
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : DomainObject, ITechnologicalParameter
{
    public virtual void Configure(EntityTypeBuilder<T> builder)
    {
        // 1. B4=C,,:C ;DÄ=D´9 C,,=CD5C=A CD;D0 ?D 5CD>D$2D 0D"5C08D0 4D41C´8D >C$0C08D0 ?C @C <CTBD >C" =C >C
        builder.HasIndex(x => new { x.OperationId, x.ParameterId })
            .IsUnique();
    }
}

// 2. B A0%B A0 At Bd B0 TU%' d"ÄDU" ,, D ?D 0C$;CT=C,,5 CD;D0 6ögB FVÆWFR•
// A,,AC=;DäGC 5CÄ 7C ?C,,ADÄ 8Cr 2D´1Cä@C=8, CTAC´8 D44C ;CT=C AC <C AC$0CtL (ISoftDeletable.Deleted
// A,, A, 5D ;C, 1D´; CÄ0C4:Cä CCD0C´5C0 AC < C0@C,,2D07C =C0KC´ BCTEC0>C´>C48Dt5D :C,,9 C00D 0CÄ5D$@ (P
builder.HasQueryFilter(x =>
    x.DeletedAt == null &&
    EF.Property<DateTime?>(x.Parameter, "DeletedAt") == null);
}
}
</file>

<file path="Promatis.Net.MES.Data/Configuration/TechnologicalOperationUnitBaseConfiguration.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Data;

/// <summary>
/// A=C0DC,,3D4@C FC,,0 CD;D0 1C 7D² AC$0Ct8D
/// </summary>
/// <typeparam name="T"></typeparam>
public abstract class TechnologicalOperationUnitBaseConfiguration<T, TOperation> :
    IEntityTypeConfiguration<T>
    where T : TechnologicalOperationUnitBase<TOperation> // <--- A05D 5CD0CT< D$8C0 >C05D 0Dd8C•
    where TOperation : DomainObject, ITechnologicalOperation
{
    public virtual void Configure(EntityTypeBuilder<T> builder)
    {
        // 1. B4=C,,:C ;DÄ=D´9 C,,=CD5C=A CD;D0 ?C @D² ?Cä;CT9 (A0@CT4CäBC$@C IC 5D" 4D41C´8D >C$0C08CR AC$0Ct5
        builder.HasIndex(x => new { x.OperationId, x.UnitId })
            .IsUnique();
    }
}

// 2. B A0%B A0 At Bd B0 TU%' d"ÄDU" ,, D ?D 0C$;CT=C,,5 CD;D0 6ögB FVÆWFR•
// BD8C´LD$@D45CÄ AC <D2 AC$0CtCDäID4N D CD"=CäAD$L, CTAC´8 Cä=C ?Cä<CTGCT=C :C : D44C ;CT=C00D0İ
// A,, A, 5D ;C, CCD0C´5C0> D 2D07C =C0>CR A C05C´ >C >D CCD>C$0C08CR ..Væ-B´ı
// A,,AC0>C´LctCCT< EF.Property CD;D0 1CT7Cä?C AC0>C4> CD>D BD4?C : D$5C05C$KCÄ0C$=D´< D 2Cä9D BC$0C
builder.HasQueryFilter(x =>
    x.DeletedAt == null &&
    EF.Property<DateTime?>(x.Unit, "DeletedAt") == null);
}
}
</file>

<file path="Promatis.Net.MES.Data/Configuration/UnitBaseConfiguration.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.MES.Domain;

namespace Promatis.Net.MES.Data;

public class UnitBaseConfiguration : IEntityTypeConfiguration<UnitBase>
{
    public void Configure(EntityTypeBuilder<UnitBase> builder)
    {
        // A$:C´NDt0CT< D BD 0D$5C48Dä E B 4C´O C$ACT9 C,,5D 0D EC,,8 UnitBase
        builder.UseTptMappingStrategy();
    }
}

```

```

    // A00D BD >C":C ?Cä;CT9D
    builder.Property(x => x.Kind).IsRequired();D
    builder.Property(x => x.Type).IsRequired();D
}D
}
</file>

<file path="Promatis.Net.MES.Data/DbContext/MesApplicationDbContext.cs">
using Microsoft.EntityFrameworkCore;D
using Microsoft.Extensions.Configuration;D
using Promatis.Net.Data;D
using Promatis.Net.MES.Domain;D
D
namespace Promatis.Net.MES.Data;D
D
public class MesApplicationDbContext(D
    DbContextOptions options,D
    IConfiguration configuration,D
    IServiceProvider? serviceProvider = null) D
    : ApplicationDbContext(options, configuration, serviceProvider)D
{D
    public DbSet<UnitOfMeasurement> UnitOfMeasurements => Set<UnitOfMeasurement>();D
}
</file>

<file path="Promatis.Net.MES.Data/Promatis.Net.MES.Data.csproj">
<Project Sdk="Microsoft.NET.Sdk">D
D
    <PropertyGroup>D
        <TargetFramework>net10.0</TargetFramework>D
        <ImplicitUsings>enable</ImplicitUsings>D
        <Nullable>enable</Nullable>D
    </PropertyGroup>D
D
    <ItemGroup>D
        <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.8" />D
        <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />D
        <PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />D
    </ItemGroup>D
D
    <ItemGroup>D
        <ProjectReference Include="..\..\Core\Data\Promatis.Net.Data.csproj" />D
        <ProjectReference Include="..\Promatis.Net.MES.Domain\Promatis.Net.MES.Domain.csproj" />D
    </ItemGroup>D
D
</Project>
</file>

<file path="Promatis.Net.MES.Data/Triggers/TechnologicalOperationHierarchyTrigger.cs">
using Microsoft.EntityFrameworkCore;D
using Promatis.Net.Data;D
using Promatis.Net.MES.Domain;D
D
namespace Promatis.Net.MES.Data;D
D
/// <summary>D
/// A 1D BD 0C=BC0KC' 1C 7Cä2D'9 D$@C,,3C45D 8CT@C @DT8C, BCTEC0>C'>C48Dt5D :C,,E Cä?CT@C FC,,9.D
/// At0D"8D"0CTB D BD CC=BD4@D2 >D" =CT:Cä@D 5C=BC0>C4> CD>C 0C$;CT=C,,0 C0>CDCCT;Cä2 C, :Cä=D$@Cä;C,,@D45D" AD
/// </summary>D
public abstract class TechnologicalOperationHierarchyTrigger<T, TOLink, TPLink> :
IBeforeSaveTrigger<T>D
    where T : TechnologicalOperationBase<T, TOLink, TPLink>D
    where TOLink : classD
    where TPLink : classD
{D
    public async Task HandleAsync(EntityCancelEventArgs<T> args)D
    {D
        T operation = args.Entity;D

        // ATAC'8 Cä?CT@C FC,,0 C=D =CT2C 0 - C0@Cä?D4AC=0CT< C0@Cä2CT@C=C0D
        if (!operation.ParentId.HasValue || operation.ParentId == Guid.Empty) return;D

        // A,,7C$;CT:C 5CÄ 8Cr !B4 AB AD$0D$CD -4Æv b 8 Name D >CD8D$5C'LD :Cä9 Cä?CT@C FC,,8D
        var parentInfo = await args.Context.Set<T>().D
            .Where(x => x.Id == operation.ParentId)D

```

```

        .Select(x => new { x.IsLeaf, x.Name })}
        .FirstOrDefaultAsync();
    }

    if (parentInfo != null)
    {
        // A,,ACô>C´LctCCT< CÔ0D, 4Cä<CT=CÔKC´ 4C$8Cd>C¢ 4C´O Cô@Cä2CT@C¤8 Cô@C 2C,,; C$;Cä6CT=CÔ>D BC,1
        // Aô5D 5CD0CT< D BC BD4A IsLeaf D >CD8D$5C´O, C$KD$0D"5CÔ=D´9 C,,7 C 0CtK.
        if (parentInfo.IsLeaf)
        {
            args.Cancel = true;
            args.ErrorMessage = $"Aô0D CD,,5CÔ8CR 8CT@C @DT8C, ÔU3¢ "CTECÔ>C´>C48Dt5D :C 0 Cä?CT@C FC,,0 '{
                $"CÔ5 CÄ>Cd5D" 1D´BDÄ 2C´>Cd5CÔ0 C" w· &VçD-æfðäæ ÖWÔrÂ BC : C¤0C¢ >CÔ0
            }

            return;
        }
    }
}

// AD>Cô>C´=C,,BCT;DÄ=C 0 Ct0D"8D$0 CÔ0 C,,7CÄ5CÔ5CÔ8CS¢ 5D ;C, >Cô5D 0Dd8Dò AD$0CÔ>C$8D$ADò C,,AD$>CÄÄ
// Cô@Cä2CT@Dô5CÄ 2 ChangeTracker C,,;C, ABÄ =CTB C´8 D2 =CT5 CD>Dt5D =C,,E DÔ;CT<CT=D$>C-
if (operation.IsLeaf)
{
    bool hasChildrenInDb = await args.Context.Set<T>()
        .AnyAsync(x => x.ParentId == operation.Id);

    if (!TechnologicalOperationHierarchyEngine.CanChangeLeafStatus(hasChildrenInDb,
        operation.IsLeaf))
    {
        args.Cancel = true;
        args.ErrorMessage = $"Aô0D CD,,5CÔ8CR FCT;CäAD$=CäAD$8 CD0CÔ=D´E: Aô5C$>Ct<Cä6CÔ> D 4CT;C BDÄ
            $"D$0C¢ :C : C$=D4BD 8 CÔ5CR CCD5 D >CD5D 6C BD 0 CD@D43C,,5 D$5DT=Cä;Cä3C
        }
    }
}
}
</file>

<file path="Promatis.Net.MES.Data/Triggers/UnitBaseHierarchyTrigger.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.MES.Domain;

namespace Promatis.Net.MES.Data;

/// <summary>
/// B$@C,,3C45D 4C´O Cô@Cä2CT@C¤8 D ?CTFC,,DC,,GCÔKDR ?D 0C$8C² 8CT@C @DT8C, Væ-D& 6Rí
/// </summary>
public class UnitBaseHierarchyTrigger : IBeforeSaveTrigger<UnitBase>
{
    public async Task HandleAsync(EntityCancelEventArgs<UnitBase> args)
    {
        UnitBase unit = args.Entity;

        if (!unit.ParentId.HasValue || unit.ParentId == Guid.Empty) return;

        // Aô>C´CDt0CT< Kind C, æ ÖR @Cä4C,,BCT;Dò 4C´O C,,=DD>D <C BC,,2CÔ>C4> D >Cä1D"5CÔ8Dý
        var parentInfo = await args.Context.Set<UnitBase>()
            .Where(x => x.Id == unit.ParentId)
            .Select(x => new { x.Kind, x.Name })
            .FirstOrDefaultAsync();

        if (parentInfo != null && !UnitHierarchyEngine.IsHierarchyValid(parentInfo.Kind, unit.Kind))
        {
            args.Cancel = true;
            args.ErrorMessage = $"Aô0D CD,,5CÔ8CR 8CT@C @DT8C, ÔU3¢ C JCT:D" w·Væ-Bäæ ÖWÔr ‡·Væ-Bä¶-æGò' " ½
                $"CÔ5 CÄ>Cd5D" 1D´BDÄ 2C´>Cd5CÔ 2 '{parentInfo.Name}' ({parentInfo.Kind}).";
        }
    }
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Enums/CalculationMethod.cs">
using System.ComponentModel;

namespace Promatis.Net.MES.Domain.Interface;

/// <summary>
/// AÄ5D$>CDK D 0D GCTBC BCTE. Cô0D 0CÄ5D$@Cä2D

```

```

/// </summary>
[Flags]
public enum CalculationMethod
{
    [Description("A05 Ct0CD0C0"•Ý
    None = 0, // 00000000
}
    [Description("AÄ0C=AC,,<C ;DÄ=Cä5 Ct=C GCT=C,,5 (MAX)")]
    Max = 1,
}
    [Description("AÄ8C08CÄ0C´LC0>CR 7C00Dt5C08CR „0"â'"•Ý
    Min = 2,
}
    [Description("B @CT4C05CR 7C00Dt5C08CR „ dr'"•Ý
    Avg = 4,
}
    [Description("A05D 2Cä5 Ct=C GCT=C,,5 (FIRST)")]
    First = 8,
}
    [Description("A0>D ;CT4C05CR 7C00Dt5C08CR „Ä 5B'"•Ý
    Last = 16,
}
    [Description("A$ACR 7C00Dt5C08D0 „ ÄÄ'"•Ý
    All = 32
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Enums/UnitKind.cs">
using System.ComponentModel;
}
namespace Promatis.Net.MES.Domain.Interface;
}
public enum UnitKind
{
    [Description("B :C´0CDAC=0D0 ;Cä3C,,AD$8C=0")]
    Storage = UnitType.Storage | UnitType.Zone | UnitType.Rack | UnitType.Cell | UnitType.Crane,
}
    [Description("A0@Cä8Ct2Cä4D BC$5C0=C 0 Ct>C00")]
    Production = UnitType.Workshop | UnitType.Section | UnitType.Line | UnitType.Workstation |
UnitType.MachineTool | UnitType.Table,
}
    [Description("B$@C =D ?Cä@D$=D´9 D47CT;")]
    Transport = UnitType.Vehicle | UnitType.Conveyor,
}
    [Description("A0>CD@C 7CD5C´5C08CR"•Ý
    Department = UnitType.Workshop | UnitType.Section | UnitType.Other,
}
    [Description("B 0C >Dt0D0 BCäGC=0 / B0GCT9C=0")]
    Position = UnitType.Cell | UnitType.Other
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Enums/UnitType.cs">
using System.ComponentModel;
}
namespace Promatis.Net.MES.Domain.Interface;
}
[Flags]
public enum UnitType
{
    None = 0,
}
    [Description("Bd5DR"•Ý
    Workshop = 1,
}
    [Description("B4GC AD$>CΦ"•Ý
    Section = 2,
}
    [Description("A´8C08D0 0 Cä=C$5C"5D "•Ý
    Line = 4,
}
    [Description("B 0C >Dt5CR <CTAD$>")]
    Workstation = 8,
}
    [Description("B :C´0CB"•Ý
    Storage = 16,
}

```

```

    [Description("At>CÔ0 DT@C =CT=C,,0"))]
    Zone = 32,
    [Description("B BCT;C´0Cb"•Ý
    Rack = 64,
    [Description("BôGCT9C@0 C 4D 5D 0 DT@C =CT=C,,0"))]
    Cell = 128,
    [Description("A@C = / Aô>CDJCT<CÔ8C¢"•Ý
    Crane = 256,
    [Description("B BC =Cä:")]
    MachineTool = 512,
    [Description("A$5D AD$0C¢ ò !D$>C²"•Ý
    Table = 1024,
    [Description("B$@C =D ?Cä@D$=Cä5 D @CT4D BC$>")]
    Vehicle = 2048,
    [Description("A 2D$>CÔ>CÄ=D´9 D$@C =D ?Cä@D$5D "•Ý
    Conveyor = 4096,
    [Description("Aô@CäGCT5")]
    Other = 8192
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalOperation.cs">
namespace Promatis.Net.MES.Domain.Interface;
{
    public interface ITechnologicalOperation
    {
    }
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalParameter.cs">
namespace Promatis.Net.MES.Domain.Interface;
{
    public interface ITechnologicalParameter
    {
    }
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Promatis.Net.MES.Domain.Interface.csproj">
<Project Sdk="Microsoft.NET.Sdk">
{
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <ProjectReference Include="..\..\Core\Domain.Interface\Promatis.Net.Domain.Interface.csproj" />
    </ItemGroup>
}
</Project>
</file>

<file path="Promatis.Net.MES.Domain/Promatis.Net.MES.Domain.csproj">
<Project Sdk="Microsoft.NET.Sdk">
{
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <ProjectReference Include="..\..\Core\Domain\Promatis.Net.Domain.csproj" />
        <ProjectReference Include="..

```

```

\Promatis.Net.MES.Domain.Interface\Promatis.Net.MES.Domain.Interface.csproj" />
</ItemGroup>
</Project>
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBase.cs">
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Domain;

/// <summary>
/// A 0Ct>C$0D0 BCTEC0>C'>C48Dt5D :C 0 Cä?CT@C FC,,0.
/// A00D ;CT4D45D" GC,,AD$>CR =CR04Cd5C05D 8C¢ 4CT@CT2Câ !B4 ABÂ CC 8D 0D0 :Cä=DD;C,,:D$K CÄ0C0?C,,=C40,
/// C, ?D 5CD>D BC 2C'0CTB D BD >C4> D$8C08Ct8D >C$0C0=D'9 C,,=D$5D DCT9D 4C'0 C 8Ct=CTA-C'>C48C¤8.D
/// </summary>
public abstract class TechnologicalOperationBase<T, TOLink, TPLink> : ReferenceTreeBase<T>, // <- A¤ A,, "A,, 'AT
    ITechnologicalOperation
    where T : TechnologicalOperationBase<T, TOLink, TPLink>
    where TOLink : class
    where TPLink : class
{
    public bool IsLeaf { get; set; } = true;

    public virtual ICollection<TOLink> UnitLinks { get; set; } = new List<TOLink>();
    public virtual ICollection<TPLink> ParameterLinks { get; set; } = new List<TPLink>();
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Domain;

// A$0C'8CD0D$>D >C05D 0Dd8C, ,,>D BC 5D$AD0 1CT7 C,,7CÄ5C05C08C'•
public abstract class TechnologicalOperationBaseValidator<T> : ReferenceTreeBaseValidator<T>
    where T : ReferenceTreeBase<T>, ITreeNode<T>, ITechnologicalOperation // <- A¤ A,, "A,, 'AT!A¤ AR B B A$ A
{
    protected TechnologicalOperationBaseValidator() : base()
    {
        // A 0Ct>C$K¢' &Vfw&Væ6T& 6Uf Æ-F F÷" CCd5 C0@Cä2CT@C,,; Cä1D"8CR ?Cä;D0 ,,BÂ æ ÖR•
        // At4CTADÄ <D² ?C,,HCT< C0@C 2C,,;C AD$@Cä3Cä 4C'0 D$5DT=Cä;Cä3C,,GCTAC¤8DR >C05D 0Dd8C•

        RuleFor(x => x.Code)
            .NotEmpty()
            .WithMessage("A¤>CB >C05D 0Dd8C, >C 0Ct0D$5C'5C0â"•
            .MaximumLength(50)
            .WithMessage("A¤>CB >C05D 0Dd8C, =CR 4Cä;Cd5C0 ?D 5C$KD,,0D$L 50 D 8CÄ2Cä;Cä2.");
    }
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperation/
TechnologicalOperationHierarchyEngine.cs">
using System.Reflection;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Domain;

public static class TechnologicalOperationHierarchyEngine
{
    /// <summary>
    /// A4;Cä1C ;DÄ=D'9 C,,AD$>Dt=C,,: C0@C 2CDK: C0@Cä2CT@D05D"Â 2C ;C,,4C0> C'8 C$;Cä6CT=C,,5 CD>Dt5D =CT9 Cä?C
    /// A00CÄ5D BC$> C ;Cä:C,,@D45D" ACä7CD0C08CR 2CTBCä: C$=D4BD 8 D$5D <C,,=C ;DÄ=D'E Cä?CT@C FC,,9 (IsLeaf =
    /// B$8C08Ct0Dd8D0 ?CT@CT2CT4CT=C =C GC,,AD$K¢' :Cä=D$@C :D" 4CT@CT2C BÄ GD$> C0>C'=CäAD$LDâ CD BD 0C00
    /// </summary>
    /// <typeparam name="T">A¤>C0:D 5D$=D'9 D$8C0 BCTEC0>C'>C48Dt5D :Cä9 Cä?CT@C FC,,8</typeparam>
    /// <param name="parent">B >CD8D$5C'LD :C 0 Cä?CT@C FC,,0</param>
    /// <param name="childIsLeaf">A0@C,,7C00C¢ BCä3CäÂ 0C$;D05D$AD0 ;C, ACä7CD0C$0CT<D'9/C05D 5CÄ5D"0CT<D'9 D
    public static bool IsHierarchyValid<T>(T? parent, bool childIsLeaf)
    where T : class, ITreeNode<T>
    {

```



```

// 1. ATAC'8 D >CD8D$5C'0 C05D"Â <D² ACä7CD0CT< C¤>D =CT2Cä9 D0;CT<CT=D" r MD$> C$ACT3CD0 C$0C'8CD=Cä
if (parent == null)0
{0
    return true;0
}0
0
// 2. Bt5D 5Cr @CTDC'5C¤AC,,N (C,,;C, :C AD" : C,,=D$5D DCT9D C, CTAC'8 C K Cä= C KC²' 8ct2C'5C¤0CT< C0@
// B$0C¢ :C : T C00D ;CT4D45D$AD0 >D" FV6†æöÆöv-6 Ä÷ W& F-öä& 6RÄ AC$>C"AD$2Cä -4ÆV b BC < C40D 0C0BC
PropertyInfo? isLeafProperty = typeof(T).GetProperty("IsLeaf");0
if (isLeafProperty != null)0
{0
    bool parentIsLeaf = (bool)isLeafProperty.GetValue(parent)!;0
    if (parentIsLeaf)0
    {0
        return false; // B$5D <C,,=C ;DÄ=D'9 D47CT; C05 CÄ>Cd5D" 1D'BDÄ @Cä4C,,BCT;CT<0
    }0
}0
0
return true;0
}0
0
/// <summary>0
/// A0@Cä2CT@D05D"Â @C 7D 5D,,5C0> C'8 C05D 5C¤;DäGC BDÄ DC'0C2 -4ÆV b C D CD"5D BC$CDäICT9 Cä?CT@C FC,,8.0
/// ATAC'8 D2 >C05D 0Dd8C, CCd5 DD8Ct8Dt5D :C, 5D BDÄ 4CäGCT@C08CR 2CTBC¤8 C" At#, C0@CT2D 0D"0D$L CTQ C
/// </summary>0
public static bool CanChangeLeafStatus(bool hasChildren, bool requestedIsLeaf)0
{0
    if (hasChildren && requestedIsLeaf)0
    {0
        return false;0
    }0
}0
0
return true;0
}0
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalOperationParameter/
TechnologicalOperationParameterBase.cs">
using Promatis.Net.Domain;0
using Promatis.Net.Domain.Interface;0
using Promatis.Net.MES.Domain.Interface;0
0
namespace Promatis.Net.MES.Domain;0
0
/// <summary>0
/// B 2D07D4ND"0D0 AD4IC0>D BDÄ ' C0>C48CR0:Cä0<C0>C48CĚ² <CT6CDC D$5DT=Cä;Cä3C,,GCTAC¤8CÄ8 Cä?CT@C FC,,0CÄ8 C,
/// Aä?D 5CD5C'0CTB D$@CT1Cä2C =C,,0 C¢ 7C ?Cä;C05C08Dä ?C @C <CTBD >C" 4C'0 C¤>C0:D 5D$=Cä9 Cä?CT@C FC,,8.0
/// </summary>0
public abstract class TechnologicalOperationParameterBase<T0peration, TParameter> : DomainObject,
ISoftDeletable0
where T0peration : DomainObject, ITechnologicalOperation0
where TParameter : DomainObject, ITechnologicalParameter0
{0
    public Guid OperationId { get; set; }0
0
    /// <summary>0
    /// A00C$8C40Dd8Cä=C0>CR AC$>C"AD$2Cä : C 0Ct>C$>C' >C05D 0Dd8C,1
    /// </summary>0
    public virtual T0peration Operation { get; set; } = null!;0
0
    public Guid ParameterId { get; set; }0
0
    /// <summary>0
    /// A00C$8C40Dd8Cä=C0>CR AC$>C"AD$2Cä : C 0Ct>C$>CÄC C00D 0CÄ5D$@D2i
    /// </summary>0
    public virtual TParameter Parameter { get; set; } = null!;0
0
    /// <summary>0
    /// A0@C,,7C00C¢ BCä3CäÂ GD$> C00D 0CÄ5D$@ Cä1D07C BCT;CT= CD;D0 7C ?Cä;C05C08D0i
    /// </summary>0
    public bool IsRequired { get; set; } = false;0
0
    /// <summary>0
    /// A0>CÄ8C00C'LC0>CR 7C00Dt5C08CR ?C @C <CTBD 0 C0> D4<Cä;Dt0C08Däi
    /// </summary>0

```

```

        public string? DefaultValue { get; set; }
    }
    /// <summary>
    /// AÄ8C08CÄ0C´LCÖ> CD>CöCD BC,,<Cä5 Ct=C GCT=C,,5.
    /// </summary>
    public double? MinValue { get; set; }
}
    /// <summary>
    /// AÄ0C=AC,,<C ;DÄ=Cä 4Cä?D4AD$8CÄ>CR 7CÖ0Dt5CÖ8CRí
    /// </summary>
    public double? MaxValue { get; set; }
}
    public DateTime? DeletedAt { get; set; }
    public string? DeletedBy { get; set; }
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperationParameter/
TechnologicalOperationParameterBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain.Interface;
}
namespace Promatis.Net.MES.Domain;
}
public abstract class TechnologicalOperationParameterBaseValidator<T, TOperation, TParameter> :
DomainObjectValidator<T>
    where T : TechnologicalOperationParameterBase<TOperation, TParameter>
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
{
    protected TechnologicalOperationParameterBaseValidator()
    {
        RuleFor(x => x.OperationId)
            .NotEmpty()
            .WithMessage("A,,4CT=D$8DD8C=0D$>D >Cö5D 0Dd8C, >C 0Ct0D$5C´5CÖâ""½
    }
    RuleFor(x => x.ParameterId)
        .NotEmpty()
        .WithMessage("A,,4CT=D$8DD8C=0D$>D BCTECÖ>C´>C48Dt5D :Cä3Cä ?C @C <CTBD 0 Cä1Dö7C BCT;CT=.");
    // A= @CäAD Ö?Cä;CT2C 0 C$0C´8CD0Dd8Döf C :D 8CÄ0C´LCÖ>CR 7CÖ0Dt5CÖ8CR =CR <Cä6CTB C KD$L CÄ5CÖLD,,5 C
    RuleFor(x => x.MaxValue)
        .GreaterThanOrEqualTo(x => x.MinValue)
        .When(x => x.MinValue.HasValue && x.MaxValue.HasValue)
        .WithMessage("AÄ0C=AC,,<C ;DÄ=Cä 4Cä?D4AD$8CÄ>CR 7CÖ0Dt5CÖ8CR =CR <Cä6CTB C KD$L CÄ5CÖLD,,5 CÄ8CÖ8C
    }
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperationUnit/TechnologicalOperationUnitBase.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain.Interface;
}
namespace Promatis.Net.MES.Domain;
}
/// <summary>
/// A 1D BD 0C=BCÖ0Dò 1C 7C 4C´O D 2Dö7C, ,,@C AD,,8D OCT<C 0)>
/// </summary>
public abstract class TechnologicalOperationUnitBase<TOperation> : DomainObject, ISoftDeletable
    where TOperation : DomainObject, ITechnologicalOperation
{
    public Guid OperationId { get; set; }
}
    /// <summary>
    /// AÖ0C$8C40Dd8Cä=CÖ>CR AC$>C"AD$2Cä : Cä?CT@C FC,,8 (C CCD5D" 7C :D KD$> C= >CÖ:D 5D$=D´< D$8Cö>CÄ 2 MDM)
    /// </summary>
    public virtual TOperation Operation { get; set; } = null!;
}
    public Guid UnitId { get; set; }
}
    /// <summary>
    /// Aö@Dö<C 0 D AD´;C=0 CÖ0 C 0Ct>C$KC´ :C´0D A Cä1Cä@D44Cä2C =C,,0. D
    /// EF Core C 2D$>CÄ0D$8Dt5D :C, AC$0Cd5D" MD$> D BC 1C´8Dd5C´ >C >D CCD>C$0CÖ8Dò ?Cä ACä3C´0D,,5CÖ8Dö<.D
    /// </summary>

```

```

    public virtual UnitBase Unit { get; set; } = null!;
}
public int Priority { get; set; } = 1;
public DateTime? DeletedAt { get; set; }
public string? DeletedBy { get; set; }
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperationUnit/
TechnologicalOperationUnitBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain.Interface;
namespace Promatis.Net.MES.Domain;
/// <summary>
/// A 0Ct>C$KC' 2C ;C,,4C BCä@ D 2Dô7C•
/// </summary>
public abstract class TechnologicalOperationUnitBaseValidator<T, TOperation> :
DomainObjectValidator<T>
where T : TechnologicalOperationUnitBase<TOperation>
where TOperation : DomainObject, ITechnologicalOperation
{
    protected TechnologicalOperationUnitBaseValidator()
    {
        RuleFor(x => x.OperationId)
            .NotEmpty()
            .WithMessage("A,,4CT=D$8DD8C=0D$>D >Cô5D 0Dd8C, >C 0Ct0D$5C'5C0â""½
D
        RuleFor(x => x.UnitId)
            .NotEmpty()
            .WithMessage("A,,4CT=D$8DD8C=0D$>D >C >D CCD>C$0C08Dò >C 0Ct0D$5C'5C0â""½
D
        RuleFor(x => x.Priority)
            .InclusiveBetween(1, 10)
            .WithMessage("Aô@C,,>D 8D$5D" >C >D CCD>C$0C08Dò 4Cä;Cd5C0 1D'BDÂ 2 CD8C ?C 7Cä=CR >D" 4Cä â""
    }
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBase.cs">
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain.Interface;
namespace Promatis.Net.MES.Domain;
/// <summary>
/// A 0Ct>C$KC' BCTECô>C'>C48Dt5D :C,,9 C00D 0CÄ5D$@ (D ?D 0C$>Dt=C,,: DT0D 0C=0Ct@C,,AD$8C¢ ?D >Dd5D ACä2).D
/// </summary>
public abstract class TechnologicalParameterBase : ReferenceBase, ITechnologicalParameter
{
    /// <summary>
    /// A$=CTHC08C' :C'Ndr =C ACô@C 2CäGC08C¢ 5CD8C08Db 8Ct<CT@CT=C,,0.D
    /// </summary>
    public Guid? UnitOfMeasurementId { get; set; }
D
    public virtual UnitOfMeasurement? UnitOfMeasurement { get; set; }
D
    /// <summary>
    /// B$8Cò 4C =C0KDR ?C @C <CTBD 0 (C00C0@C,,<CT@: Bt8D ;CâÂ !D$@Cä:C Â D4;CT2Câ'í
    /// </summary>
    public string DataType { get; set; } = "Numeric";
D
    /// <summary>
    /// B 0Ct@CTHCT=C0KCR <CTBCä4D² @C AD GE BC
    /// </summary>
    public CalculationMethod AllowedMethods { get; set; } = CalculationMethod.None;
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;
D

```

```

namespace Promatis.Net.MES.Domain;
public abstract class TechnologicalParameterBaseValidator<T> : ReferenceBaseValidator<T>
    where T : TechnologicalParameterBase
{
    private static readonly string[] AllowedDataTypes = ["Numeric", "String", "Boolean",
"DateTime"];
    protected TechnologicalParameterBaseValidator() : base()
    {
        // 1. A$0C'8CD0Dd8D0 5CD8C08DdK C,,7CÄ5D 5C08D0 „4CäAD$CC0=C Ä BC : C=0C¢ B r MD$> TechnologicalParame
RuleFor(x => x.UnitOfMeasurement)
        .NotNull()
        .WithMessage("AT4C,,=C,,FC 8Ct<CT@CT=C,,0 C05 CÄ>Cd5D" 1D'BDÄ çVÆÄâ""½
    }
    // 2. A$0C'8CD0Dd8D0 BC,,?C 4C =C0KD]
RuleFor(x => x.DataType)
        .NotEmpty()
        .WithMessage("B$8C0 4C =C0KDR ?C @C <CTBD 0 Cä1D07C BCT;CT=.")
        .Must(type => AllowedDataTypes.Contains(type))
        .WithMessage($"A05CD>C0CD BC,,<D'9 D$8C0 4C =C0KDRâ C 7D 5D,,5C0=D'5 Ct=C GCT=C,,0: {string.Join(",
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/
TechnologicalParameterCalcMethodBase.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain.Interface;
namespace Promatis.Net.MES.Domain;
public abstract class TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter> :
DomainObject, ISoftDeletable
    where TUnit : UnitBase
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
{
    public Guid UnitId { get; set; }
    /// <summary>
    /// Bd5DT>C$0D0 5CD8C08Dd0
    /// </summary>
    public virtual TUnit Unit { get; set; } = null!;
    public Guid TechnologicalOperationId { get; set; }
    /// <summary>
    /// B$5DT=Cä;Cä3C,,GCTAC=0D0 >C05D 0Dd8Dý
    /// </summary>
    public virtual TOperation TechnologicalOperation { get; set; } = null!;
    public Guid TechnologicalParameterId { get; set; }
    /// <summary>
    /// B$5DT=Cä;Cä3C,,GCTAC=8C' ?C @C <CTBD
    /// </summary>
    public virtual TParameter TechnologicalParameter { get; set; } = null!;
    /// <summary>
    /// AÄ5D$>CB @C Adt5D$0
    /// </summary>
    public CalculationMethod CalculationMethod { get; set; } = CalculationMethod.None;
    public DateTime? DeletedAt { get; set; }
    public string? DeletedBy { get; set; }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/
TechnologicalParameterCalcMethodBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain.Interface;

```

```

namespace Promatis.Net.MES.Domain;
{
    public abstract class TechnologicalParameterCalcMethodBaseValidator<T, TUnit, TOperation, TParameter> : DomainObjectValidator<T>
    {
        where T : TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter>
        where TUnit : UnitBase
        where TOperation : DomainObject, ITechnologicalOperation
        where TParameter : TechnologicalParameterBase

        protected TechnologicalParameterCalcMethodBaseValidator()
        {
            RuleFor(x => x.UnitId)
                .NotEmpty().WithMessage("A,,4CT=D$8DD8C=D$>D FCTECä2Cä9 CT4C,,=C,,FD² >C OCT0D$5C´5C0â""½

            RuleFor(x => x.TechnologicalOperationId)
                .NotEmpty().WithMessage("A,,4CT=D$8DD8C=D$>D BCTECÔ>C´>C48Dt5D :Cä9 Cä?CT@C FC,,8 Cä1Dô7C BCT;CT=

            RuleFor(x => x.TechnologicalParameterId)
                .NotEmpty().WithMessage("A,,4CT=D$8DD8C=D$>D BCTECÔ>C´>C48Dt5D :Cä3Câ ?C @C <CTBD 0 Cä1Dô7C BCT;

            // A0@Cä2CT@Dô5CÂÂ GD$> CÄ5D$>CB @C ADt5D$0 C KC² OC$=Câ 2D´1D 0C0 ?Cä;DÄ7Cä2C BCT;CT<D
            RuleFor(x => x.CalculationMethod)
                .NotEqual(CalculationMethod.None)
                .WithMessage("A05Cä1DT>CD8CÄ> C$KC @C BDÂ :Cä@D 5C=BC0KC´ <CTBCä4 D 0D GCTBC ?C @C <CTBD >C"â""½

        }
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameterValue/TechnologicalParameterValueBase.cs">
using Promatis.Net.Domain;
{
    namespace Promatis.Net.MES.Domain;
    {
        public abstract class TechnologicalParameterValueBase<TUnit, TParameter> : DomainObject
        {
            where TUnit : UnitBase
            where TParameter : TechnologicalParameterBase

            public Guid UnitId { get; set; }

            /// <summary>
            /// Bd5DT>C$0Dò 5CD8C08Dd0 (C,,AD$>Dt=C,,: CD0C0=D´E)
            /// </summary>
            public virtual TUnit Unit { get; set; } = null!;

            public Guid TechnologicalParameterId { get; set; }

            /// <summary>
            /// B$5DT=Cä;Cä3C,,GCTAC=8C´ ?C @C <CTBD
            /// </summary>
            public virtual TParameter TechnologicalParameter { get; set; } = null!;

            /// <summary>
            /// B KD >CR , $3D OCT=Cä5") Ct=C GCT=C,,5, C05D 5CD0C0=Cä5 CäB SCADA/IoT C,,;C, >C05D 0D$>D 0D
            /// </summary>
            public string Value { get; set; } = string.Empty;

            /// <summary>
            /// B$>Dt=Cä5 C$@CT<Dò DC,,:D 0Dd8C, ?C @C <CTBD 0 (Time-Series CÄ5D$:C •
            /// </summary>
            public DateTime Date { get; set; } = DateTime.UtcNow;
        }
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameterValue/
TechnologicalParameterValueBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;
{
    namespace Promatis.Net.MES.Domain;
    {
        public abstract class TechnologicalParameterValueBaseValidator<T, TUnit, TParameter> :
        DomainObjectValidator<T>
        {
            where T : TechnologicalParameterValueBase<TUnit, TParameter>
            where TUnit : UnitBase
            where TParameter : TechnologicalParameterBase

        }
    }
}

```

```

protected TechnologicalParameterValueBaseValidator()
{
    RuleFor(x => x.UnitId)
        .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D FCTECä2Cä9 CT4C,,=C,,FD² >C OCT0D$5C'5C0â""½
D
    RuleFor(x => x.TechnologicalParameterId)
        .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D BCTECÔ>C'>C48Dt5D :Cä3Cä ?C @C <CTBD 0 Cä1Dô7C BCT;
D
    RuleFor(x => x.Value)
        .NotNull().WithMessage("At=C GCT=C,,5 C00D 0CÄ5D$@C =CR <Cä6CTB C KD$L null.")
        // B 0Ct@CTHC 5CÄ ?D4AD$CDâ AD$@Cä:D2Ä BC : C=0C¢ 4C'0 String-C00D 0CÄ5D$@Cä2 D0BCä <Cä6CTB C KD$
        // C0> D41C,,@C 5CÄ AC'CDt0C"=D'5 C0@Cä1CT;D² ?Cä :D 0D0<D
        .Must(val => val == null || val.Trim() == val)
        .WithMessage("At=C GCT=C,,5 C05 CÄ>Cd5D" =C GC,,=C BDÄAD0 8C'8 Ct0C=0C0GC,,2C BDÄAD0 ?D >C 5C'0CÄ8."
D
    RuleFor(x => x.Date)
        .Must(date => date <= DateTime.UtcNow.AddSeconds(5))
        .WithMessage("A$@CT<D0 DC,,:D 0Dd8C, ?C @C <CTBD 0 C05 CÄ>Cd5D" 1D'BDÂ 2 C CCDCD"5CÄâ""½
    }
}
</file>

<file path="Promatis.Net.MES.Domain/UnitBase/UnitBase.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain.Interface;
D
namespace Promatis.Net.MES.Domain;
D
/// <summary>D
/// A 1D BD 0C=BC00D0 1C 7C 4C'0 C$ACTE Cä@C40C08Ct0Dd8Cä=C0KDR 8 C0@Cä8Ct2Cä4D BC$5C0=D'E CT4C,,=C,,F.D
/// </summary>D
public abstract class UnitBase : ReferenceTreeBase<UnitBase>, IAudit
{
    public UnitKind Kind { get; init; }
    public required UnitType Type { get; set; }
}
</file>

<file path="Promatis.Net.MES.Domain/UnitBase/UnitBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;
D
namespace Promatis.Net.MES.Domain;
D
/// <summary>D
/// A 1D BD 0C=BC0KC' 2C ;C,,4C BCä@ CD;D0 1C 7Cä2Cä3Cä :C'0D AC Væ-D& 6Rı
/// </summary>D
/// <typeparam name="T">B$8C0Ä =C AC'5CDCCT<D'9 CäB UnitBase.</typeparam>D
public abstract class UnitBaseValidator<T> : ReferenceBaseValidator<T>
    where T : UnitBase
{
    protected UnitBaseValidator() : base()
    {
        // A0@Cä2CT@C=0, DtBCä 2 Type C05 C0@C,,HC'> C0CD BCä5/CD5DD>C'BC0>CR 7C00Dt5C08CR DC'0C4>C-
        RuleFor(x => x.Type).NotEmpty();
D
        // A0@Cä2CT@C=0 D >CäBC$5D$AD$2C,,0 A=0D$5C4>D 8C, ,,¶-æB' 8 B$8C00 (Type) C00 CäAC0>C$5 C 8D$>C$KDR <C
        RuleFor(x => x)
            .Must(u => ((int)u.Kind & (int)u.Type) != 0)
            .WithMessage("B$8C0 =CR ACä>D$2CTBD BC$CCTB C=0D$5C4>D 8C,â""½
    }
}
</file>

<file path="Promatis.Net.MES.Domain/UnitBase/UnitHierarchyEngine.cs">
using Promatis.Net.MES.Domain.Interface;
D
namespace Promatis.Net.MES.Domain;
D
public static class UnitHierarchyEngine
{
    /// <summary>D
    /// A4;Cä1C ;DÄ=D'9 C,,AD$>Dt=C,,: C0@C 2CDK: C0@Cä2CT@D05D"Ä 2C ;C,,4C0> C'8 C$;Cä6CT=C,,5 D 5C 5C0:C 2 D >
    /// A,,AC0>C'LCtCCTBD 0 C00CÄ5D BC$> C=0C¢ 2 B #A -D$@C,,3C45D 0DRÄ BC : C, 2 UI-C$0C'8CD0Dd8C,ı
    /// </summary>D

```

```

public static bool IsHierarchyValid(UnitKind parentKind, UnitKind childKind)
{
    // 1. B$5D <C,,=C ;DÄ=D´9 D47CT; Position C08C=C44C =CR <Cä6CTB C KD$L D >CD8D$5C´5Cí
    if (parentKind == UnitKind.Position) return false;

    // 2. AD5C00D BC <CT=D" <Cä6CTB D >CD5D 6C BDÂ 2D Q, C@Cä<CR ÷6–F–öí
    if (parentKind == UnitKind.Department) return childKind != UnitKind.Position;

    // 3. B ?CTFC,,0C´8Ct8D >C$0C0=D´5 Ct>CÔK C,,7Cä;C,,@Cä2C =D² 4D CC2 >D" 4D CC40, C0> CÄ>C4CD" ACä4CT@C
    if (parentKind is UnitKind.Production or UnitKind.Transport or UnitKind.Storage)
    {
        return childKind == parentKind || childKind == UnitKind.Position;
    }

    return true;
}

/// <summary>
/// A$>Ct2D 0D"0CTB D ?C,,ACä: Aä Aä B A0 B´% D$8C0>C" ...Væ–EG– R´ 4C´O C$KC00CD0DäICT3Cä AC08D :C 2 UI,D
/// CäAC0>C$KC$0D0ADÄ =C C BCT3Cä@C,,8 (UnitKind) D >CD8D$5C´LD :Cä3Cä CCT;C Ä 2 C@>D$>D KC´ <D² ACT9Dt0
/// </summary>
public static IEnumerable<UnitType> GetAllowedChildTypes(UnitKind parentKind)
{
    var allTypes = (UnitType[])Enum.GetValues(typeof(UnitType));
    var allowedTypes = new List<UnitType>();

    foreach (UnitType type in allTypes)
    {
        if (type == UnitType.None) continue;

        // 1. B =C GC ;C =C ECä4C,,<, C¢ :C :Cä<D2 ?CäBCT=Dd8C ;DÄ=Cä<D2 ¶–æB >D$=CäAC,,BD 0 D0BCäB D$8C0í
        // B :C =C,,@D45CÄ 2D 5 C@0D$5C4>D 8C,Ä GD$>C K C0>C00D$L, C" :C :D4N CÄ0D :D2 ?Cä?C 4C 5D" MD$>D"
        foreach (UnitKind potentialChildKind in (UnitKind[])Enum.GetValues(typeof(UnitKind)))
        {
            // ATAC´8 D$8C0 ?Cä1C,,BCä2Cä 2DT>CD8D" 2 D0BD2 :C BCT3Cä@C,,N0
            if (((int)potentialChildKind & (int)type) != 0)
            {
                // A, 5D ;C, ?D 0C$8C´0 C,,5D 0D EC,,8 B At AT(A .B" 2C´>Cd8D$L D0BD2 :C BCT3Cä@C,,N C" BCT
                if (IsHierarchyValid(parentKind, potentialChildKind))
                {
                    if (!allowedTypes.Contains(type))
                    {
                        allowedTypes.Add(type);
                    }
                }
            }
        }
    }

    return allowedTypes;
}

```

```

<file path="Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurement.cs">
using Promatis.Net.Domain;
namespace Promatis.Net.MES.Domain;
/// <summary>
/// AD>CÄ5C0=C 0 D CD"=CäAD$L CT4C,,=C,,FD² 8Ct<CT@CT=C,,0 (A0!A,' 4C´O D$5DT=Cä;Cä3C,,GCTAC=8DR ?C @C <CTBD >C"
/// </summary>
public class UnitOfMeasurement : ReferenceBase
{
    // B 2Cä9D BC$0 Id, Code, Name, Description C00D$8C$=Cä CC00D ;CT4Cä2C =D² >D" &VfW&Væ6T& 6Rí
    // Code – C@C BC@>CR >C >Ct=C GCT=C,,5 (C00C0@C,,<CT@, "°C", "CÄ<", "Cä1/CÄ8C0"´í
    // Name – C0>C´=Cä5 C00C,,<CT=Cä2C =C,,5 (C00C0@C,,<CT@, "A4@C 4D4A Bd5C´LD 8Dò"Ä $ C,,;C´8CÄ5D$@").
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurementValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;
namespace Promatis.Net.MES.Domain;

```

```

public class UnitOfMeasurementValidator : ReferenceBaseValidator<UnitOfMeasurement>
{
    // A05D 5Cä?D 5CD5C`OCT< C 0Ct>C$CDâ @CT3D4;D0@C=C C00 D 0D HC,,@CT=C0CDâ ,,@D4A/C`0D"Â FC,,DD K, C0@Cä1CT;D
    protected override string CodeRegexPattern => @"^[\p{L}\p{N}_\.\^\°\/\*\-\s\%]*$";
}

protected override string CodeValidationMessage => "A0@C BC0>CR >C >Ct=C GCT=C,,5 CÄ>Cd5D" ACä4CT@Cd0D$L C

public UnitOfMeasurementValidator() : base()
{
    // A0@C 2C,,;Cä æ÷DV× G' 8 Matches D46CR >D$@C 1CäBC ND" 2 C 0Ct>C$>CÄ :C`0D ACR ?Cä =C HCT<D2 =Cä2Cä<

    RuleFor(x => x.Code)
        .NotEmpty().WithMessage("A0@C BC0>CR >C >Ct=C GCT=C,,5 Cä1D07C BCT;DÄ=Cä 4C`O Ct0C0>C`=CT=C,,0.")
        .MaxLength(15).WithMessage("A0@C BC0>CR >C >Ct=C GCT=C,,5 C05 CD>C`6C0> C0@CT2D`HC BDÄ R AC,,<

    RuleFor(x => x.Name)
        .MaxLength(100).WithMessage("A0>C`=Cä5 C00C,,<CT=Cä2C =C,,5 CT4C,,=C,,FD² 8Ct<CT@CT=C,,0 C05 CD>C`

}
}
</file>

<file path="Promatis.Net.MES.Service/Promatis.Net.MES.Service.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <ProjectReference Include="..\..\Core\Service\Promatis.Net.Service.csproj" />
        <ProjectReference Include="..\Promatis.Net.MES.Data\Promatis.Net.MES.Data.csproj" />
    </ItemGroup>
</Project>
</file>

<file path="Promatis.Net.MES.Service/TechnologicalOperationParameterService/
ITechnologicalOperationParameterService.cs">
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;

namespace Promatis.Net.MES.Service;

public interface ITechnologicalOperationParameterService<T, TOperation, TParameter> :
IBaseService<T, Guid>
    where T : TechnologicalOperationParameterBase<TOperation, TParameter>
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
{
    /// <summary>
    /// A0>C`CDt8D$L C$ACR ?C @C <CTBD K, C0@C,,2D07C =C0KCR : C0>C0:D 5D$=Cä9 Cä?CT@C FC,,8D
    /// </summary>
    Task<List<T>> GetByOperationIdAsync(Guid operationId);

    /// <summary>
    /// A0>C`CDt8D$L D$>C`LC0> Cä1D07C BCT;DÄ=D`5 C00D 0CÄ5D$@D² 4C`O Cä?CT@C FC,,8D
    /// </summary>
    Task<List<T>> GetRequiredParametersByOperationIdAsync(Guid operationId);
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalOperationParameterService/
TechnologicalOperationParameterService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;

namespace Promatis.Net.MES.Service;

public abstract class TechnologicalOperationParameterService<T, TOperation, TParameter, TContext>(

```



```

        IDbContextFactory<TContext> contextFactory)&
        : BaseService<T, Guid, TContext>(contextFactory), ITechnologicalOperationParameterService<T,
TOperation, TParameter>&
        where T : TechnologicalOperationParameterBase<TOperation, TParameter>&
        where TOperation : DomainObject, ITechnologicalOperation&
        where TParameter : TechnologicalParameterBase&
        where TContext : DbContext&
    {&
        public async Task<List<T>> GetByOperationIdAsync(Guid operationId)&
        {&
            await using TContext context = await ContextFactory.CreateDbContextAsync();&

            return await context.Set<T>().&
                .Where(x => x.OperationId == operationId)&
                .Include(x => x.Parameter) // B @C 7D2 ?Cä4D$0C48C$0CT< D ?D 0C$>Dt=C,,: C00D 0CÄ5D$@Cä2 CD;D0 T•
                .AsNoTracking()&
                .ToListAsync();&
        }&
    }&

    public async Task<List<T>> GetRequiredParametersByOperationIdAsync(Guid operationId)&
    {&
        await using TContext context = await ContextFactory.CreateDbContextAsync();&

        return await context.Set<T>().&
            .Where(x => x.OperationId == operationId && x.IsRequired)&
            .Include(x => x.Parameter)&
            .AsNoTracking()&
            .ToListAsync();&
    }&
}
</file>

```

```

<file path="Promatis.Net.MES.Service/TechnologicalOperationService/
ITechnologicalOperationService.cs">
using Promatis.Net.Domain;&
using Promatis.Net.MES.Domain;&
using Promatis.Net.MES.Domain.Interface;&
using Promatis.Net.Service;&
&
namespace Promatis.Net.MES.Service;&
&
public interface ITechnologicalOperationService<T, TLink> : ITreeService<T>&
    where T : ReferenceTreeBase<T>, ITechnologicalOperation,
Promatis.Net.Domain.Interface.IDomainObjectHasKey<Guid>&
    where TLink : class&
{&
    /// <summary>&
    /// A0>C´CDt8D$ L D ?C,,ACä: D 0Ct@CTHCT=C0>C4> Cä1Cä@D44Cä2C =C,,0 (Dä=C,,BCä2) CD;D0 :Cä=Cp@CTBC0>C´ BCTEC0
    /// </summary>&
    Task<List<UnitBase>> GetAllowedUnitsAsync(Guid operationId);&
}
</file>

```

```

<file path="Promatis.Net.MES.Service/TechnologicalOperationService/
TechnologicalOperationService.cs">
using Microsoft.EntityFrameworkCore;&
using Promatis.Net.Domain;&
using Promatis.Net.Domain.Interface;&
using Promatis.Net.MES.Domain;&
using Promatis.Net.MES.Domain.Interface;&
using Promatis.Net.Service;&
&
namespace Promatis.Net.MES.Service;&
&
public abstract class TechnologicalOperationService<T, TLink, TContext>(IDbContextFactory<TContext>
contextFactory)&
    : ReferenceTreeService<T, TContext>(contextFactory), ITechnologicalOperationService<T, TLink>&
    where T : ReferenceTreeBase<T>, ITechnologicalOperation, IDomainObjectHasKey<Guid>, new()&
    where TLink : TechnologicalOperationUnitBase<T>&
    where TContext : DbContext&
{&
    public async Task<List<UnitBase>> GetAllowedUnitsAsync(Guid operationId)&
    {&
        await using TContext context = await ContextFactory.CreateDbContextAsync();&

        // LINQ-Ct0C0@CäA CäBD 0C 0D$KC$0CTB C,,4CT0C´LC0>:D
    }&
}

```

```

// A ;C 3Cä4C @Dò BCä<D2Â GD$> C" FV6†æÖÆöv-6 Ä÷ W& F-öäVæ-D& 6R AC$>C"AD$2Câ Væ-B
// C,,<CT5D" AD$@Cä3C,,9 D$8Cò Væ-D& 6RÂ Tb 6÷&R ?CäAD$@Cä8D" GC,,AD$KC' 5 ÂÔ¼ô"â 0C$BCä<C BC,,GCTAC¼8.D
return await context.Set<TLink>()
    .Where(x => x.OperationId == operationId)
    .Select(x => x.Unit)
    .AsNoTracking()
    .ToListAsync();
}
}
/// <summary>
/// AäAD$0C$;Dô5CÂ <CTBCä4 C 1D BD 0C¼BCôKCÂ =C BCT:D4ICT< D4@Cä2C05. A¼>Cô:D 5D$=D'9 Cò@C,,:C'0CD=Cä9 D
/// D$5DT?D >Dd5D ACä2 D 0CÂ ?CT@CT>Cò@CT4CT;C,,B CT3Cä 4C'0 D >Ct4C =C,,0 D 2Cä8DR ?Cä;C,,<Cä@DD=D'E Cä?CT@
/// </summary>
public abstract override Task<T> CreateChildTemplateAsync(T parent);
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
ITechnologicalParameterCalcMethodService.cs">
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;
}
namespace Promatis.Net.MES.Service;
}
public interface ITechnologicalParameterCalcMethodService<T, TUnit, TOperation, TParameter> :
IBaseService<T, Guid>
    where T : TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter>
    where TUnit : UnitBase
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
{
    /// <summary>
    /// Aô0C"BC, :Cä=C¼@CTBCôCDâ 8CôAD$@D4:Dd8Dâ @C ADt5D$0 CD;Dò BD >C":Cf C >D CCD>C$0Cô8CR ² Cò5D 0Dd8D
    /// </summary>
    Task<T?> GetCalcMethodAsync(Guid unitId, Guid operationId, Guid parameterId);
}
}
/// <summary>
/// Aô>C'CDt8D$L C$ACR <CTBCä4D² @C ADt5D$0 CD;Dò :Cä=C¼@CTBCô>C' >Cò5D 0Dd8C, =C >C >D CCD>C$0Cô8C•
/// </summary>
    Task<List<T>> GetMethodsByOperationAndUnitAsync(Guid operationId, Guid unitId);
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
TechnologicalParameterCalcMethodService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;
}
namespace Promatis.Net.MES.Service;
}
public abstract class TechnologicalParameterCalcMethodService<T, TUnit, TOperation, TParameter,
TContext>(
    IDbContextFactory<TContext> contextFactory)
    : BaseService<T, Guid, TContext>(contextFactory), ITechnologicalParameterCalcMethodService<T,
TUnit, TOperation, TParameter>
    where T : TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter>
    where TUnit : UnitBase
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
    where TContext : DbContext
{
    public async Task<T?> GetCalcMethodAsync(Guid unitId, Guid operationId, Guid parameterId)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>()
            .FirstOrDefaultAsync(x => x.UnitId == unitId &&
                x.TechnologicalOperationId == operationId &&
                x.TechnologicalParameterId == parameterId);
    }
}
}

```

```

        public async Task<List<T>> GetMethodsByOperationAndUnitAsync(Guid operationId, Guid unitId)
        {
            await using TContext context = await ContextFactory.CreateDbContextAsync();

            return await context.Set<T>()
                .Where(x => x.TechnologicalOperationId == operationId && x.UnitId == unitId)
                .Include(x => x.TechnologicalParameter) // Aô>CD3D CCd0CT< D 2Dô7C =CôKC' ?C @C <CTBD
                .AsNoTracking()
                .ToListAsync();
        }
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Service/TechnologicalParameterService/
ITechnologicalParameterService.cs">
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public interface ITechnologicalParameterService<T> : IReferenceService<T>
    where T : TechnologicalParameterBase
{
    /// <summary>
    /// Aô>C´CDt8D$L C00D 0CÄ5D$@D²Â >D$DC,,;DÄBD >C$0CÔ=D´5 C0> D$8C0C CD0CÔ=D´E (Numeric, String, Boolean, D
    /// </summary>
    Task<List<T>> GetByDataTypeAsync(string dataType);
}
</file>

```

```

<file path="Promatis.Net.MES.Service/TechnologicalParameterService/
TechnologicalParameterService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public abstract class TechnologicalParameterService<T, TContext>(IDbContextFactory<TContext>
contextFactory)
    : ReferenceService<T, TContext>(contextFactory), ITechnologicalParameterService<T>
    where T : TechnologicalParameterBase
    where TContext : DbContext
{
    public async Task<List<T>> GetByDataTypeAsync(string dataType)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();

        return await context.Set<T>()
            .Where(x => x.DataType == dataType)
            .AsNoTracking()
            .ToListAsync();
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Service/UnitOfMeasurementService/IUnitOfMeasurementService.cs">
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
/// <summary>
/// Aô>CÔBD 0C▯B D 5D 2C,,ACô>C4> D ;Cä0 CD;Dò CCô@C 2C´5C08Dò ACô@C 2CäGC08C▯>CÄ 5CD8C08Db 8Ct<CT@CT=C,,0.D
/// </summary>
public interface IUnitOfMeasurementService : IReferenceService<UnitOfMeasurement>
{
}
</file>

```

```

<file path="Promatis.Net.MES.Service/UnitOfMeasurementService/UnitOfMeasurementService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.MES.Data;
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;

```

```

namespace Promatis.Net.MES.Service;
{
    /// <summary>
    /// B 5D 2C,,A D4?D 0C$;CT=C,,0 CÔ>D <C BC,,2CÔ>-D ?D 0C$>Dt=Cä9 C,,=DD>D <C FC,,5C' 5CD8C08Db 8Ct<CT@CT=C,,0.D
    /// </summary>
    public class UnitOfMeasurementService(IDbContextFactory<MesApplicationDbContext> contextFactory)
        : ReferenceService<UnitOfMeasurement, MesApplicationDbContext>(contextFactory),
        IUnitOfMeasurementService
    {
    }
}
</file>

<file path="Promatis.Net.MES.Service/UnitService/IUnitBaseService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;
{
namespace Promatis.Net.MES.Service;
{
public interface IUnitBaseService<TContext> : IReferenceTreeService<UnitBase, TContext> // <- A,,!Aô A A' Aô
D$8C00 C" @Cä4C,,BCT;DÄAC=8C' 8C0BCT@DD5C"A!D
where TContext : DbContext
{
    Task<List<UnitBase>> GetByKindAsync(UnitKind kind);
    Task<List<UnitBase>> GetByTypeAsync(UnitType type);
    Task<List<UnitBase>> GetByKindAndTypeAsync(UnitKind kind, UnitType type);
}
</file>

<file path="Promatis.Net.MES.Service/UnitService/UnitBaseService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;
{
namespace Promatis.Net.MES.Service;
{
public abstract class UnitBaseService<TContext>(IDbContextFactory<TContext> contextFactory)
    : ReferenceTreeService<UnitBase, TContext>(contextFactory), IUnitBaseService<TContext>
    where TContext : DbContext
{
    // =====
    // Aô A "BD B AT Aô+A' A !B$ A B$ B' BT#Aç ,, AT A' At#AT"B / Aô B Aä Aô Ad )D
    // =====
    {
        /// <summary>
        /// A=C0BD 0C=B DD0C @C,,:C,â !C <C @CT0C'8Ct0Dd8D0 1D44CTB C00C08D 0C00 C00 D ;Cä9 C08Cd5 D
        /// C" :Cä=C@CTBC0>CÄ ACT@C$8D 5, C=D$>D KC' 2C,,4C,,B C0>C'8CÄ>D DC0KCR :C'0D AD² ,,FW 'FÖVçEVæ-B 8 CD@.
        /// </summary>
        public abstract override Task<UnitBase> CreateChildTemplateAsync(UnitBase parent);
    }
    // =====
    // Bt B "B' AÄ B$ AD+ A$+A B A, A Aô+BR ,,!C$>C"AD$2C ¶-æB 8 Type D$5C05D L C$8CD=D² 8CD5C ;DÄ=Câ•
    // =====
    {
        public async Task<List<UnitBase>> GetByKindAsync(UnitKind kind)
        {
            await using TContext context = await ContextFactory.CreateDbContextAsync();
            return await context.Set<UnitBase>()
                .AsNoTracking()
                .Where(x => x.Kind == kind)
                .ToListAsync();
        }
    }
    {
        public async Task<List<UnitBase>> GetByTypeAsync(UnitType type)
        {
            await using TContext context = await ContextFactory.CreateDbContextAsync();
            // A,,AC0>C'LCtCCT< C0>C 8D$>C$>CR $ ", D$0Cç :C : UnitType - D0BCä ´fÆ w5Y
            return await context.Set<UnitBase>()
                .AsNoTracking()
                .Where(x => (x.Type & type) != 0)
                .ToListAsync();
        }
    }
    {
        public async Task<List<UnitBase>> GetByKindAndTypeAsync(UnitKind kind, UnitType type)
    }
}
}
}

```

```

        {
            await using TContext context = await ContextFactory.CreateDbContextAsync();
            return await context.Set<UnitBase>().
                .AsNoTracking().
                .Where(x => x.Kind == kind && (x.Type & type) != 0).
                .ToListAsync();
        }
    }
}
</file>

<file path="Promatis.Net.MES.Tests/AssemblyInfo.cs">
// AÄBC;DäGC 5D" ?C @C ;C´5C´LCÔ>CR 2D´?Cä;CÔ5CÔ8CR BCTAD$>C" 2 DÔBCä9 D 1Cä@Cä5D
[assembly: CollectionBehavior(DisableTestParallelization = true)]
</file>

<file path="Promatis.Net.MES.Tests/Promatis.Net.MES.Tests.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
        <IsPackable>false</IsPackable>
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="coverlet.collector" Version="10.0.0">
            <PrivateAssets>all</PrivateAssets>
            <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</
IncludeAssets>
        </PackageReference>
        <PackageReference Include="FluentAssertions" Version="8.10.0" />
        <PackageReference Include="FluentValidation" Version="12.1.1" />
        <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="10.0.8" />
        <PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />
        <PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.5.1" />
        <PackageReference Include="Moq" Version="4.20.72" />
        <PackageReference Include="xunit.runner.visualstudio" Version="3.1.5">
            <PrivateAssets>all</PrivateAssets>
            <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</
IncludeAssets>
        </PackageReference>
        <PackageReference Include="xunit.v3" Version="3.2.2" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Promatis.Net.Mes.Service\Promatis.Net.MES.Service.csproj" />
    </ItemGroup>
    <ItemGroup>
        <Using Include="Xunit" />
    </ItemGroup>
</Project>
</file>

<file path="Promatis.Net.MES.Tests/Service/UnitServiceStandaloneTests.cs">
using FluentAssertions;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.ChangeTracking;
using Microsoft.Extensions.Configuration;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.MES.Service;
namespace Promatis.Net.MES.Tests;
public class UnitServiceStandaloneTests
{
    // 1. A„ACô>C´LctCCT< C$5Ct4CR FW7DF$6öçFW‡M
    private readonly IDbContextFactory<TestDbContext> _factory;
    private readonly DbContextOptions<TestDbContext> _options;

```

```

public class TestUnit : Domain.UnitBase { }
}

/// <summary>
/// B$5D BCä2D'9 D 5D 2C,,A CD;Dò 2CT@C,,DC,,:C FC,,8 C 8Ct=CTA-C$KC >D >C¢ Væ-D& 6R 2 CÄ>CDCC'LCÔKDR BCTAD$0
/// A,,!Aô A A' Aô : B 8C4=C BD4@C ?D 8C$5CD5C00 C¢ AD$0C04C @D$C UnitBaseService D >CD=C,,< generic-C00
/// </summary>
public class TestUnitService(IDbContextFactory<TestDbContext> factory)
    : UnitBaseService<TestDbContext>(factory)
{
    /// <summary>
    /// Aô@CäAD$5C"HC O Ct0C4;D4HC=0 DD0C @C,,:C, ?Cä;C,,<Cä@DD8Ct<C 4C'0 D4AC05D,,=Cä9 C= >CÄ?C,,;DôFC,,8 C 1
    /// </summary>
    public override Task<Domain.UnitBase> CreateChildTemplateAsync(Domain.UnitBase parent)
    {
        // A,,!Aô A A' Aô : A 5Ct>C00D =Cä5 CD8C00CÄ8Dt5D :Cä5 D >Ct4C =C,,5 C,,=D BC =D 0 C00D$8C$=Cä3Cä B
        var childMock = (Domain.UnitBase)Activator.CreateInstance(parent.GetType());

        childMock.ParentId = parent.Id;
        childMock.Parent = null;

        return Task.FromResult(childMock);
    }
}

public UnitServiceStandaloneTests()
{
    // 2. Aä?Dd8C, 4Cä;Cd=D² 1D'BDÂ 4C'0 TestDbContext
    _options = new DbContextOptionsBuilder<TestDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .Options;

    // 3. AÄ>C=0CT< DD0C @C,,:D2 8CÄ5C0=Cä 4C'0 TestDbContext
    var factoryMock = new Mock<IDbContextFactory<TestDbContext>>();

    factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))
        .ReturnsAsync(() => new TestDbContext(_options)); // ReturnsAsync D4?D >D"0CTB Task.FromResult

    _factory = factoryMock.Object;
}

// 4. A= >C0BCT:D B D$>Cd5 CD>C'6CT= C0@C,,=C,,<C BDÂ ?D 0C$8C'LCÔKCR >C0FC,,8D
public class TestDbContext(DbContextOptions<TestDbContext> options)
    : ApplicationDbContext(options, new ConfigurationBuilder().Build())
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<Domain.UnitBase>().UseTptMappingStrategy();
        modelBuilder.Entity<TestUnit>().ToTable("TestUnits");
    }

    public override async Task<int> SaveChangesAsync(CancellationToken cancellationToken =
default)
    {
        IEnumerable<EntityEntry<Domain.UnitBase>> entries =
ChangeTracker.Entries<Domain.UnitBase>()
        .Where(e => e.State is EntityState.Added or EntityState.Modified);

        foreach (EntityEntry<Domain.UnitBase> entry in entries)
        {
            Domain.UnitBase unit = entry.Entity;
            if (unit.ParentId.HasValue)
            {
                // At0C4@D46C 5CÄ @Cä4C,,BCT;Dò 4C'0 C0@Cä2CT@C=8 (C" -äÖVö+' f-æB @C 1CäBC 5D" 1D'AD$@Cä
                Domain.UnitBase? parent = Set<Domain.UnitBase>().Find(unit.ParentId.Value);
                if (parent != null)
                {
                    // 1. Aô@C 2C,,;Cä ÷6-F-öâ ,, "CT@CÄ8C00C'LCÔKC' Cct5C²•
                    if (parent.Kind == UnitKind.Position)
                        throw new InvalidOperationException("Position node cannot have
children.");
                }

                // 2. Aô@C 2C,,;Cä FW 'FÖVçB ,, CR <Cä6CTB D >CD5D 6C BDÂ ÷6-F-öâ•
                if (parent.Kind == UnitKind.Department && unit.Kind == UnitKind.Position)
                    throw new InvalidOperationException($"A00D CD,,5C08CR 8CT@C @DT8Cf¢ >C JCT:D" :C B
'{parent.Kind}'.");
            }
        }
    }
}

```

```

        // 3. Aô@C 2C,,;Câ 8Ct>C´ODd8C, ... &öGV7F-ôâö7F÷& vRöG& ç7 ÷'B =CR ACÄ5D,,8C$0DäBD O)Ð
        if (parent.Kind is UnitKind.Production or UnitKind.Storage or
UnitKind.Transport)Ð
        {Ð
            if (unit.Kind != parent.Kind && unit.Kind != UnitKind.Position)Ð
                throw new InvalidOperationException($"Aô0D CD,,5C08CR 8CT@C @DT8Cfç w·Væ-Bä¶-æ
'{parent.Kind}'.");Ð
            }Ð
        }Ð
    }Ð
}Ð

    return await base.SaveChangesAsync(cancellationToken);Ð
}Ð
}Ð
}Ð
}Ð

[Fact]Ð
public async Task GetByKindAsync_Should_Return_Units_Matching_Complex_Mask()Ð
{Ð
    // ArrangeÐ
    var service = new TestUnitService(_factory);Ð

    // BôGCT9C=0 C$ECä4C,,B C" <C AC=C StorageÐ
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Cell_01", Kind =
UnitKind.Storage, Type = UnitType.Cell });Ð
    // B BC =Cä: C$ECä4C,,B C" <C AC=C ProductionÐ
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Lathe_01", Kind =
UnitKind.Production, Type = UnitType.MachineTool });Ð

    // ActÐ
    List<Domain.UnitBase> storageUnits = await service.GetByKindAsync(UnitKind.Storage);Ð

    // AssertÐ
    storageUnits.Should().ContainSingle(x => x.Name == "Cell_01");Ð
    storageUnits.Should().NotContain(x => x.Name == "Lathe_01");Ð
}Ð

[Fact]Ð
public async Task GetByTypeAsync_Should_Handle_Specific_Flags()Ð
{Ð
    // ArrangeÐ
    var service = new TestUnitService(_factory);Ð
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Crane_1", Kind =
UnitKind.Storage, Type = UnitType.Crane });Ð
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Vehicle_1", Kind =
UnitKind.Transport, Type = UnitType.Vehicle });Ð

    // ActÐ
    List<Domain.UnitBase> result = await service.GetByTypeAsync(UnitType.Crane);Ð

    // AssertÐ
    result.Should().ContainSingle(x => x.Name == "Crane_1");Ð
}Ð

[Fact]Ð
public async Task MoveAsync_Should_Throw_When_Moving_Into_Position()Ð
{Ð
    // ArrangeÐ
    var service = new TestUnitService(_factory);Ð
    var posId = Guid.NewGuid();Ð
    var machineId = Guid.NewGuid();Ð

    await service.AddAsync(new TestUnit { Id = posId, Name = "Terminal Point", Kind =
UnitKind.Position, Type = UnitType.Other });Ð
    await service.AddAsync(new TestUnit { Id = machineId, Name = "Mobile Machine", Kind =
UnitKind.Production, Type = UnitType.MachineTool });Ð

    // Act & AssertÐ
    Func<Task> act = async () => await service.MoveAsync(machineId, posId);Ð

    await act.Should().ThrowAsync<InvalidOperationException>()Ð
        .WithMessage("Position node cannot have children.");Ð
}Ð

```

```

ð
[Fact]ð
public async Task MoveAsync_Should_Prevent_Circular_Dependencies()ð
{ð
    // Arrangeð
    var service = new TestUnitService(_factory);ð
    var idA = Guid.NewGuid();ð
    var idB = Guid.NewGuid();ð
ð
    // AD>C 0C$;D05CÂ ¶-æB 8 Type, D$0C¢ :C : Cä=C, >C 0Ct0D$5C´LC0K (required / init)ð
    await service.AddAsync(new TestUnitð
    {ð
        Id = idA,ð
        Name = "Node A",ð
        Kind = UnitKind.Department,ð
        Type = UnitType.Workshopð
    });ð
ð
    await service.AddAsync(new TestUnitð
    {ð
        Id = idB,ð
        Name = "Node B",ð
        ParentId = idA,ð
        Kind = UnitKind.Production,ð
        Type = UnitType.Sectionð
    });ð
ð
    // Act & Assertð
    Func<Task> act = async () => await service.MoveAsync(idA, idB);ð
ð
    await act.Should().ThrowAsync<InvalidOperationException>()ð
        .WithMessage("*Bd8C¢;C,,GCTAC¢0D0 7C 2C,,AC,,<CäAD$L*");ð
    }ð
ð
[Fact]ð
public async Task GetFullTreeAsync_Should_Maintain_Hierarchy_And_References()ð
{ð
    // Arrangeð
    var service = new TestUnitService(_factory);ð
    var rootId = Guid.NewGuid();ð
    var sectionId = Guid.NewGuid();ð
ð
    await service.AddAsync(new TestUnitð
    {ð
        Id = rootId,ð
        Name = "Dept",ð
        Kind = UnitKind.Department,ð
        Type = UnitType.Workshopð
    });ð
ð
    await service.AddAsync(new TestUnitð
    {ð
        Id = sectionId,ð
        Name = "Section",ð
        Kind = UnitKind.Production,ð
        Type = UnitType.Section,ð
        ParentId = rootIdð
    });ð
ð
    await service.AddAsync(new TestUnitð
    {ð
        Id = Guid.NewGuid(),ð
        Name = "Workstation",ð
        Kind = UnitKind.Production,ð
        Type = UnitType.Workstation,ð
        ParentId = sectionIdð
    });ð
ð
    // Actð
    Domain.UnitBase? tree = await service.GetFullTreeAsync(rootId);ð
ð
    // Assertð
    tree.Should().NotBeNull();ð
    tree!.Children.Should().HaveCount(1);ð
    tree.Children.First().Children.Should().HaveCount(1);ð
    }ð

```



```

    [Fact]
    public async Task MoveAsync_Department_Should_Not_Contain_Position()
    {
        // Arrange
        var service = new TestUnitService(_factory);
        var deptId = Guid.NewGuid();
        var positionId = Guid.NewGuid();

        // 1. B >Ct4C 5CÂ CT?C @D$0CÄ5C0B
        await service.AddAsync(new TestUnit
        {
            Id = deptId,
            Name = "Main Dept",
            Kind = UnitKind.Department,
            Type = UnitType.Workshop
        });

        // 2. B >Ct4C 5CÂ Cä7C,,FC,,N (D >C4;C AC0> D$2Cä5C' <C AC5, D0BCä <Cä6CTB C KD$L Cell)
        await service.AddAsync(new TestUnit
        {
            Id = positionId,
            Name = "Work Point 1",
            Kind = UnitKind.Position,
            Type = UnitType.Cell
        });

        // Act
        // A0KD$0CT<D 0 C05D 5CÄ5D BC,,BDÂ ÷6-F-0â 2 Department
        Func<Task> act = async () => await service.MoveAsync(positionId, deptId);

        // Assert
        // Aä6C,,4C 5CÂ >D,,8C :D2 >D" Væ-D& 6T†-W& &6‡•G&-vvW-
        await act.Should().ThrowAsync<InvalidOperationException>()
            .WithMessage($"*C=0D$5C4>D 8C, u ÷6-F-0âr =CR <Cä6CTB C KD$L C$;Cä6CT= C" tFW 'F0VçBrç""½

    }

    [Fact]
    public async Task MoveAsync_Production_Should_Contain_Production_Subnode()
    {
        // Arrange
        var service = new TestUnitService(_factory);
        var parentProdId = Guid.NewGuid();
        var childProdId = Guid.NewGuid();

        await service.AddAsync(new TestUnit
        {
            Id = parentProdId,
            Name = "Production Area",
            Kind = UnitKind.Production,
            Type = UnitType.Workshop
        });

        await service.AddAsync(new TestUnit
        {
            Id = childProdId,
            Name = "Machine Node",
            Kind = UnitKind.Production,
            Type = UnitType.MachineTool
        });

        // Act
        Func<Task> act = async () => await service.MoveAsync(childProdId, parentProdId);

        // Assert
        // At4CTADÂ 8D :C´Ndt5C08D0 1D´BDÂ =CR 4Cä;Cd=CâÂ BC : C00C0 &0GV7F-0â <Cä6CTB D >CD5D 6C BDÂ &0GV7
        await act.Should().NotThrowAsync();

        Domain.UnitBase? updated = await service.GetByIdAsync(childProdId);
        updated!.ParentId.Should().Be(parentProdId);
    }
}
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/AppDbContext.cs">
using Microsoft.EntityFrameworkCore;

```

```

Ð
namespace Promatis.Net.MES.Tests.UnitBase;Ð
Ð
// B$5D BCä2D´9 C¤>CÔBCT:D B C$=D4BD 8 Cö@CäAD$@C =D BC$0 C„<CT= D$5D BCä2D
public class AppDbContext : DbContextÐ
{Ð
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }Ð
Ð
    // A„ACö>C´LCtCCT< C 0Ct>C$KC´ BC„?, DtBCä1D² BD 8C43CT@ CÄ>C2 4CT;C BDÂ 6WCÄVæ–D& 6Sâ,•
    public DbSet<Domain.UnitBase> Units => Set<Domain.UnitBase>();Ð
Ð
    protected override void OnModelCreating(ModelBuilder modelBuilder)Ð
    {Ð
        // 1. Aô0D BD 0C„2C 5CÂ E B „:C : C" >D =Cä2CÔ>CÂ ?D >CT:D$5)Ð
        modelBuilder.Entity<Domain.UnitBase>().UseTptMappingStrategy();Ð
Ð
        // 2. B4:C 7D´2C 5CÂÂ GD$> TestUnit – DôBCâ =C AC´5CD=C„:Ð
        modelBuilder.Entity<TestUnit>().ToTable("TestUnits");Ð
    }Ð
}
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/TestUnit.cs">
namespace Promatis.Net.MES.Tests.UnitBase;Ð
Ð
public class TestUnit : MES.Domain.UnitBase { }
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/TestUnitValidator.cs">
using Promatis.Net.MES.Domain;Ð
Ð
namespace Promatis.Net.MES.Tests.UnitBase;Ð
Ð
public class TestUnitValidator : UnitBaseValidator<TestUnit> { }
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/UnitBaseArchitectureTests.cs">
using FluentAssertions;Ð
using FluentValidation.TestHelper;Ð
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Data;Ð
using Promatis.Net.MES.Data;Ð
using Promatis.Net.MES.Domain.Interface;Ð
Ð
namespace Promatis.Net.MES.Tests.UnitBase;Ð
Ð
public class UnitBaseArchitectureTestsÐ
{Ð
    private readonly TestUnitValidator _validator = new();Ð
    private readonly DbContextOptions<AppDbContext> _options;Ð
Ð
    public UnitBaseArchitectureTests()Ð
    {Ð
        _options = new DbContextOptionsBuilder<AppDbContext>().Ð
            .UseInMemoryDatabase(Guid.NewGuid().ToString())Ð
            .Options;Ð
    }Ð
Ð
    private static EntityCancelEventArgs<MES.Domain.UnitBase> CreateArgs(MES.Domain.UnitBase
entity, DbContext context)Ð
    {Ð
        return new EntityCancelEventArgs<MES.Domain.UnitBase>(Ð
            entity,Ð
            EntityStateChangeEnum.Added,Ð
            new List<PropertyChangeInfo>(),Ð
            context);Ð
    }Ð
Ð
    #region A$0C´8CD0Dd8Dò „ C AC¤8 C, 1C 7Cä2D´5 Cö@C 2C„;C •
Ð
    [Theory]Ð
    [InlineData(UnitKind.Storage, UnitType.Cell, true)] // BôGCT9C¤0 C" !C¤;C 4CR r C-
    [InlineData(UnitKind.Storage, UnitType.Workshop, false)] // Bd5DR 2 B :C´0CD5 – AâHC„1C¤0 (C 8D" =CR ACä2
    [InlineData(UnitKind.Production, UnitType.Table, true)] // B BCä; C" D >C„7C$>CDAD$2CR r C-
    [InlineData(UnitKind.Position, UnitType.Other, true)] // Aô@CäGCT5 C" Cä7C„FC„8 – Aä:Ð
    [InlineData(UnitKind.Position, UnitType.Table, false)] // B BCä; C" Cä7C„FC„8 – AâHC„1C¤0 (D >C4;C ACô>

```

```

    public void Validator_Should_Check_Bitwise_Compatibility(UnitKind kind, UnitType type, bool
isInvalid)ð
    {ð
        var unit = new TestUnit { Kind = kind, Type = type, Name = "Test" };ð
        TestValidationResult<TestUnit>? result = _validator.TestValidate(unit);ð
ð
        if (isInvalid)ð
        {ð
            result.ShouldNotHaveAnyValidationErrors();ð
        }ð
        elseð
        {ð
            result.ShouldHaveValidationErrorFor(x => x)ð
                .WithErrorMessage("B$8Cò =CR ACä>D$2CTBD BC$CCTB C=0D$5C4>D 8C,â""½
        }ð
    }ð
ð
    #endregionð
ð
    #region B$@C,,3C45D ,, D 0C$8C´0 C$;Cä6CT=CÔ>D BC, ¶-æB•
ð
    [Fact]ð
    public async Task Trigger_Should_Allow_Root_Units()ð
    {ð
        await using var context = new AppDbContext(_options);ð
        var root = new TestUnit { ParentId = null, Kind = UnitKind.Department, Type =
UnitType.Other };ð
ð
        var trigger = new UnitBaseHierarchyTrigger();ð
        EntityCancelEventArgs<Domain.UnitBase> args = CreateArgs(root, context);ð
ð
        await trigger.HandleAsync(args);ð
ð
        args.Cancel.Should().BeFalse();ð
    }ð
ð
    [Fact]ð
    public async Task Trigger_Should_Reject_Position_In_Department()ð
    {ð
        await using var context = new AppDbContext(_options);ð
        var parentId = Guid.NewGuid();ð
ð
        context.Units.Add(new TestUnit { Id = parentId, Kind = UnitKind.Department, Type =
UnitType.Other, Name = "Dept" });ð
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);ð
ð
        var child = new TestUnit { ParentId = parentId, Kind = UnitKind.Position, Type =
UnitType.Other, Name = "Pos" };ð
        var trigger = new UnitBaseHierarchyTrigger();ð
        EntityCancelEventArgs<Domain.UnitBase> args = CreateArgs(child, context);ð
ð
        await trigger.HandleAsync(args);ð
ð
        args.Cancel.Should().BeTrue();ð
        args.ErrorMessage.Should().Contain("CÔ5 CÄ>Cd5D" 1D´BDÂ 2C´>Cd5Cò 2 'Dept' (Department));ð
    }ð
ð
    [Theory]ð
    [InlineData(UnitKind.Production, UnitKind.Production, false)] // Bd5DR 2 Bd5DR r C-
    [InlineData(UnitKind.Production, UnitKind.Transport, true)] // B$@C =D ?Cä@D" 2 Bd5DR r C ?D 5D"5CÔ> (
    [InlineData(UnitKind.Storage, UnitKind.Position, false)] // Aô>Ct8Dd8Dò 2 B :C´0CB r C-
    [InlineData(UnitKind.Position, UnitKind.Position, true)] // Aô>Ct8Dd8Dò 2 Aô>Ct8Dd8Dâ r C ?D 5D"5CÔ
    public async Task Trigger_Should_Enforce_Isolation(UnitKind pKind, UnitKind cKind, bool
shouldCancel)ð
    {ð
        await using var context = new AppDbContext(_options);ð
        var parentId = Guid.NewGuid();ð
ð
        // B >Ct4C 5CÂ @Cä4C,,BCT;Dò A Cò>CDECä4DòIC,,< D$8Cò>Cí
        context.Units.Add(new TestUnit { Id = parentId, Kind = pKind, Type = GetFirstType(pKind),
Name = "P" });ð
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);ð
ð
        var child = new TestUnit { ParentId = parentId, Kind = cKind, Type = GetFirstType(cKind),
Name = "C" };ð
        EntityCancelEventArgs<Domain.UnitBase> args = CreateArgs(child, context);ð

```

```

        await new UnitBaseHierarchyTrigger().HandleAsync(args);
    }
    args.Cancel.Should().Be(shouldCancel, $"C0>D$>CÄC DtBCâ 2C´>Cd5C08CR ¶4¶-æG0 2 {pKind} CD>C´6C0> C0@C
    }
    #endregion
}
private static UnitType GetFirstType(UnitKind kind)
    => Enum.GetValues<UnitType>().First(t => t != UnitType.None && ((int)kind & (int)t) != 0);
}
</file>

<file path="Promatis.Net.MES.UI/_Imports.razor">
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using MudBlazor
@using Promatis.Net.UI.Components.Workspaces
@using Promatis.Net.UI.Components.Dialogs
</file>

<file path="Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementPage.razor">
@page "/mes/mdm/units-of-measurement"
@using Promatis.Net.UI.Components.References
@inherits Microsoft.AspNetCore.Components.ComponentBase
@
@* A0@CäAD$> C$KctKC$0CT< C00D,,5 D4=C,,2CT@D 0C´LC0>CR OCD@Câ 4C´O D CD´=CäAD$8 UnitOfMeasurement *@
<ReferenceGridPage TEntity="Domain.UnitOfMeasurement" />
</file>

<file path="Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementPage.razor.cs">
using Microsoft.AspNetCore.Components;
@
namespace Promatis.Net.MES.UI.Pages.UnitOfMeasurements;
@
public partial class UnitOfMeasurementPage : ComponentBase
{
    @
}
</file>

<file path="Promatis.Net.MES.UI/Promatis.Net.MES.UI.csproj">
<Project Sdk="Microsoft.NET.Sdk.Razor">
@
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <Nullable>enable</Nullable>
        <ImplicitUsings>enable</ImplicitUsings>
    </PropertyGroup>
@
    <ItemGroup>
        <Compile Remove="UnitBase\*" />
        <Content Remove="UnitBase\*" />
        <EmbeddedResource Remove="UnitBase\*" />
        <None Remove="UnitBase\*" />
    </ItemGroup>
@
    <ItemGroup>
        <SupportedPlatform Include="browser" />
    </ItemGroup>
@
    <ItemGroup>
        <PackageReference Include="Microsoft.AspNetCore.Components.Web" Version="10.0.8" />
        <PackageReference Include="MudBlazor" Version="9.4.0" />
    </ItemGroup>
@
    <ItemGroup>
        <ProjectReference Include="..\..\Core\Configuration.Web\Promatis.Net.Configuration.Web.csproj" /
>
        <ProjectReference Include="..\..\Core\Promatis.Net.UI\Promatis.Net.UI.csproj" />
        <ProjectReference Include="..\Promatis.Net.MES.Service\Promatis.Net.MES.Service.csproj" />
    </ItemGroup>
@
</Project>

```

</file>

<file path="Promatis.Net.MES.UI/UiModule.cs">

using MudBlazor;ð

using Promatis.Net.UI;ð

ð

namespace Promatis.Net.MES.UI;ð

ð

public class UiModule : IUiModuleð

{ð

public string Name => GetType().Assembly.GetName().Name ?? "Unknown.Module";ð

ð

public IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems()ð

{ð

return new List<(string Title, string Href, string Icon, string? Group)>ð

{ð

// Að;CäAC¤8C' ACô@C 2CäGCô8Cø BCTECô>C´>C48Dt5D :C„E Cô0D 0CÄ5D\$@Cä2ð

("AT4C„=C„FD² 8Ct<CT@CT=C„0", "/mes/mdm/units-of-measurement", Icons.Material.Filled.SettingsInpu

});ð

}ð

}

</file>

<file path="Promatis.Net.MES.UI/wwwroot/exampleJsInterop.js">

// This is a JavaScript module that is loaded on demand. It can export any number ofð

// functions, and may import other JavaScript modules if required.ð

ð

export function showPrompt(message) {ð

return prompt(message, 'Type anything here');ð

}

</file>

</files>